



JETBRAINS

Help the Developer

JetBrains is here to help developers improve their coding experience. For this challenge, we are asking you to create an application or IntelliJ-based plugin with the primary purpose of making the lives of developers easier. We aim to enhance the JetBrains suite of professional development tools by creating an innovative plugin that complements our existing offerings. To deliver the best solution, get started with the following materials.

1. [Get](#) free educational license
2. Create a simple IDE plugin using the template
 1. [IDE Plugin Development Course](#) — basics about PSI and creating plugins
 2. To start implementation quickly, you can use the [IntelliJ Platform Plugin Template](#) and the [IntelliJ Platform Plugin SDK documentation](#)
3. Learn about how to use the OpenAI API, any other model, or an agent framework
 1. [OpenAI API documentation](#)
 2. [Here](#) and [here](#) you can find examples that demonstrate how to use OpenAI models inside the IntelliJ IDEA plugin
 3. If you want to structure your logic as an agent rather than a single prompt-response call, take a look at [Koog](#), JetBrains' Kotlin framework for building agents — useful if your idea needs memory, multi-step reasoning, or tool calling rather than a single API round-trip

Recommended technologies: Kotlin and Java.

Description

For this challenge, we are asking you to create your application or IntelliJ-based plugin with the primary purpose of making the lives of developers easier. Some starting points:

- Routine test creator: consider what makes this more than "call the API with the function body": does it use surrounding context, existing test conventions in the repo, or catch edge cases a generic prompt would miss?
- Refactor suggester: what does *codebase-aware* refactoring look like, versus a suggestion that ignores how the rest of the project is structured?
- Your idea: anything the team comes up with. The challenge is to create an application that will make your coding experience more comfortable.

Your application is expected to be an AI solution, (you can use the OpenAI API, another model, or an agent framework such as Koog for your task). Integrating your solution with JetBrains tools would be a great idea, but it is not a requirement. The OpenAI API key is stored in an online vault. The link to it is inside your GitLab repository.

Expectations

We're mostly looking for good ideas with a proof of concept, so focus on this aspect. The final project could be:

- A plugin
- A web app
- A desktop app
- A mobile app

Judging Criteria

Your submission will be evaluated based on the following criteria (in order of priority):

1. **Implementation and Proof of Concept:** does it actually run and solve the developer pain point end-to-end, not just a mocked screenshot?
2. **User Experience and Interface:** how naturally it fits into a developer's existing workflow; friction here kills adoption even for a technically strong idea.
3. **Innovation and Creativity:** originality of the problem chosen and the angle taken, especially anything that goes beyond "wrap a prompt around a common IDE action" (see the note above).
4. **Technical Features:** depth of the solution: use of real codebase context (not just the selected snippet), multi-step or agentic logic where it's warranted, and integration with JetBrains tooling if attempted.
5. **Presentation:** clarity of the demo and explanation of design decisions.

Useful Resources for Plugins

If you consider implementing a plugin for JetBrains IDEs, we could recommend several resources:

JetBrains Academy courses:

[IDE Plugin Development Course](#) - basics about PSI and creating plugins

[Coroutines and channels](#) - a course about using coroutines in IntelliJ IDEA

To start implementation quickly, you can use [IntelliJ Platform Plugin Template](#) and IntelliJ [Platform Plugin SDK documentation](#)

[Here](#) and [here](#) you can find examples which demonstrate how to use OpenAI models inside the IntelliJ IDEA plugin

To inference ML models inside the plugin, you can use [KInference](#) library

